

**DSA0105- OBJECT ORIENTED
PROGRAMMING WITH C++**

**Smart Emergency Medical Resource Coordination
System using C++**

Submitted by:

Shahana – 192524479

Rasineni Harika – 192572140

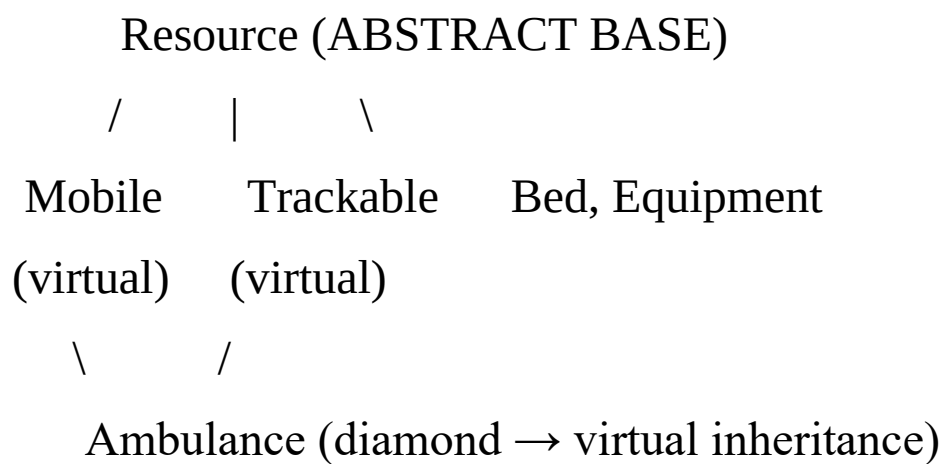
Niranjana. A – 192521370

SIMATS ENGINEERING

September,2026

Pseudocode:

1. Class Hierarchy



ResponseTeam : public Mobile

Patient (standalone)

EmergencyCase (uses Patient + Resource*)

HospitalSystem (manager class — arrays of objects, static counters)

2. Abstract Base Class- Resource

CLASS Resource (ABSTRACT)

PROTECTED:

string resourceID

```

    string type
    bool  isAvailable
    static int totalResources    // static member

PUBLIC:
    Resource()                  // default constructor
    Resource(id, type)          // parameterized
constructor
    Resource(const Resource &r)  // copy constructor
    VIRTUAL ~Resource()         // virtual destructor

    VIRTUAL float calculateUtilization() = 0    // pure
virtual
    VIRTUAL void  displayInfo() = 0             // pure
virtual

    void allocate() { isAvailable = false }
    void release()  { isAvailable = true  }
    static int getTotalResources()

END CLASS

```

3. Visual Base Branches (for diamond inheritance)

```

CLASS Mobile : VIRTUAL PUBLIC Resource
    PROTECTED:
        float speed
        string currentLocation

```

PUBLIC:

Mobile()

Mobile(speed, location)

VIRTUAL ~Mobile()

void moveTo(newLocation)

END CLASS

CLASS Trackable : VIRTUAL PUBLIC Resource

PROTECTED:

string trackingID

string status

PUBLIC:

Trackable()

Trackable(trackingID)

VIRTUAL ~Trackable()

void updateStatus(newStatus)

END CLASS

4. Diamond Class- Ambulance

CLASS Ambulance : PUBLIC Mobile, PUBLIC Trackable

PRIVATE:

int capacity

bool hasVentilator

PUBLIC:

Ambulance() // default

Ambulance(id, speed, loc, trackID, capacity) //
parameterized

Ambulance(const Ambulance &a) // copy

~Ambulance() // destructor

OVERRIDE float calculateUtilization()

OVERRIDE void displayInfo()

Ambulance operator+(const Ambulance &other) //
combine capacity

END CLASS

5. Other Resource Types:

CLASS Bed : PUBLIC Resource

PRIVATE:

string ward

bool hasOxygenSupport

PUBLIC:

Bed()

Bed(id, ward, oxygenSupport)

Bed(const Bed &b)

~Bed()

```
        OVERRIDE float calculateUtilization()
        OVERRIDE void displayInfo()
END CLASS
```

```
CLASS Equipment : PUBLIC Resource
```

```
    PRIVATE:
```

```
        string equipmentName
```

```
        int unitsAvailable
```

```
    PUBLIC:
```

```
        Equipment()
```

```
        Equipment(id, name, units)
```

```
        Equipment(const Equipment &e)
```

```
        ~Equipment()
```

```
        OVERRIDE float calculateUtilization()
```

```
        OVERRIDE void displayInfo()
```

```
END CLASS
```

```
CLASS ResponseTeam : PUBLIC Mobile
```

```
    PRIVATE:
```

```
        int teamSize
```

```
        string specialization
```

```
    PUBLIC:
```

```
        ResponseTeam()
```

```
    ResponseTeam(id, speed, loc, size, spec)
    ResponseTeam(const ResponseTeam &r)
    ~ResponseTeam()
    OVERRIDE float calculateUtilization()
    OVERRIDE void displayInfo()
END CLASS
```

6. Patient Class

```
CLASS Patient
    PRIVATE:
        string patientID
        string name
        int age
        string condition
        static int totalPatients // static member

    PUBLIC:
        Patient()
        Patient(id, name, age, condition)
        Patient(const Patient &p)
        ~Patient()

        void displayPatient()
        static int getTotalPatients()
```

END CLASS

7. EmergencyCase Class (uses operator overloading)

CLASS EmergencyCase

PRIVATE:

string caseID

Patient patient

int priorityLevel // 1 = critical ... 5 = minor

Resource* assignedResource[MAX_RESOURCES]
// array of pointers (polymorphism)

int resourceCount

bool isActive

static int activeCases // static member

PUBLIC:

EmergencyCase()

EmergencyCase(id, patient, priority)

EmergencyCase(const EmergencyCase &c)

~EmergencyCase() // releases assigned
resources

void assignResource(Resource* r)

void updateCase(newPriority)

void closeCase()


```
        BOOL operator<(const EmergencyCase &other) //  
priority comparison
```

```
        FRIEND ostream& operator<<(ostream&, const  
EmergencyCase&) // report formatting
```

```
        static int getActiveCases()
```

```
END CLASS
```

8. Manager/ Driver Class

```
CLASS HospitalSystem
```

```
    PRIVATE:
```

```
        Patient patientList[MAX]
```

```
        int patientCount
```

```
        Resource* resourcePool[MAX] // base-class pointer  
array → runtime polymorphism
```

```
        int resourceCount
```

```
        EmergencyCase caseList[MAX]
```

```
        int caseCount
```

```
    PUBLIC:
```

```
        void registerPatient()
```

```

        void registerResource()           // ask type -> create
Ambulance/Bed/Equipment/ResponseTeam via new
        void createEmergencyCase()
        void allocateResource()           // scan resourcePool for
isAvailable & suitable type
        void updateCaseStatus()
        void comparePriorities()          // uses operator
        void generateReport()             // loop resourcePool, call
virtual displayInfo() -> polymorphism
        void showMenu()

```

```

~HospitalSystem()           // delete all dynamically
allocated resources

```

END CLASS

9. Main Driver (Menu Driven Loop)

```

FUNCTION main()

```

```

    HospitalSystem system

```

```

    LOOP:

```

```

        system.showMenu()

```

```

        READ choice

```

```

        SWITCH choice:

```

```

            CASE 1: system.registerPatient()

```

```

            CASE 2: system.registerResource()

```

```
CASE 3: system.createEmergencyCase()
CASE 4: system.allocateResource()
CASE 5: system.updateCaseStatus()
CASE 6: system.comparePriorities()
CASE 7: system.generateReport()
CASE 0: PRINT "Exiting..." ; BREAK LOOP
DEFAULT: PRINT "Invalid choice"

END LOOP

END FUNCTION
```

Source Code:

```
/*
```

```
Smart Emergency Medical Resource Coordination System
DSA0105 - Object Oriented Programming with C++
```

Demonstrates:

- Classes, objects, encapsulation, access specifiers
- Static members, arrays of objects
- Default / parameterized / copy constructors, destructors
- Operator overloading (<, <<, +)
- Inheritance, virtual base class (diamond), abstract class
- Pointers, base-class pointers, virtual functions, runtime polymorphism

```
*/
```

```

#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

const int MAX_PATIENTS    = 50;
const int MAX_RESOURCES   = 50;
const int MAX_CASES       = 50;
const int MAX_CASE_RESOURCES = 5;

// ===== INPUT HELPERS
=====

int readInt(const string &prompt) {
    int value;
    while (true) {
        cout << prompt;
        cin >> value;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(10000, '\n');
            cout << "Invalid input. Please enter a whole number.\n";
            continue;
        }
    }
}

```

```

        cin.ignore(10000, '\n');
        return value;
    }
}

```

```

int readIntInRange(const string &prompt, int lo, int hi) {
    while (true) {
        int value = readInt(prompt);
        if (value < lo || value > hi) {
            cout << "Please enter a value between " << lo << " and "
<< hi << ".\n";
            continue;
        }
        return value;
    }
}

```

```

float readFloat(const string &prompt) {
    float value;
    while (true) {
        cout << prompt;
        cin >> value;
        if (cin.fail()) {
            cin.clear();

```

```

        cin.ignore(10000, '\n');
        cout << "Invalid input. Please enter a number.\n";
        continue;
    }
    cin.ignore(10000, '\n');
    return value;
}
}

```

```

string readLine(const string &prompt) {
    string value;
    cout << prompt;
    getline(cin, value);
    return value;
}

```

```

// ===== PATIENT
=====

```

```

class Patient {
private:
    string patientID;
    string name;
    int age;

```

```

    string condition;

    static int totalPatients;    // static member: total patients ever
    registered

public:
    Patient()                    // default constructor
        : patientID("NONE"), name("Unknown"), age(0),
        condition("Not specified") {}

    Patient(string id, string n, int a, string c)    //
    parameterized constructor
        : patientID(id), name(n), age(a), condition(c) {
        totalPatients++;
    }

    Patient(const Patient &p)    // copy
    constructor
        : patientID(p.patientID), name(p.name), age(p.age),
        condition(p.condition) {}

    Patient& operator=(const Patient &p) = default;    // copy
    assignment

    ~Patient() {}                // destructor

```

```
string getPatientID() const { return patientID; }  
string getName() const { return name; }
```

```
void displayPatient() const {  
    cout << "Patient ID: " << patientID  
        << " | Name: " << name  
        << " | Age: " << age  
        << " | Condition: " << condition << endl;  
}
```

```
static int getTotalPatients() { return totalPatients; }  
};  
int Patient::totalPatients = 0;
```

```
// ===== RESOURCE (ABSTRACT BASE  
CLASS) =====
```

```
class Resource {  
protected:  
    string resourceID;  
    string type;  
    bool isAvailable;  
    static int totalResources;    // static member: total resources  
ever registered
```


public:

```
    Resource()                                // default constructor  
        : resourceID("NONE"), type("Unknown"), isAvailable(true)  
    {}
```

```
    Resource(string id, string t)              // parameterized  
constructor  
        : resourceID(id), type(t), isAvailable(true) {  
        totalResources++;  
    }
```

```
    Resource(const Resource &r)                // copy  
constructor  
        : resourceID(r.resourceID), type(r.type),  
isAvailable(r.isAvailable) {}
```

```
    Resource& operator=(const Resource &r) = default;    //  
copy assignment
```

```
    virtual ~Resource() {}                        // virtual  
destructor
```

```
    virtual float calculateUtilization() const = 0;    // pure  
virtual -> abstract class
```

```
virtual void displayInfo() const = 0;           // pure
virtual -> abstract class
```

```
virtual void allocate() { isAvailable = false; }
```

```
virtual void release() { isAvailable = true; }
```

```
string getResourceID() const { return resourceID; }
```

```
string getType() const { return type; }
```

```
bool getAvailability() const { return isAvailable; }
```

```
static int getTotalResources() { return totalResources; }
```

```
};
```

```
int Resource::totalResources = 0;
```

```
// ===== VIRTUAL BASE BRANCHES
=====
```

```
class Mobile : virtual public Resource {
```

```
protected:
```

```
    float speed;
```

```
    string currentLocation;
```

```
public:
```

```
    Mobile() : speed(0), currentLocation("Unknown") {}
```

```
Mobile(float s, string loc) : speed(s), currentLocation(loc) {}  
virtual ~Mobile() {}
```

```
void moveTo(const string &newLocation) { currentLocation =  
newLocation; }
```

```
float getSpeed() const { return speed; }  
string getLocation() const { return currentLocation; }  
};
```

```
class Trackable : virtual public Resource {  
protected:
```

```
    string trackingID;  
    string status;
```

```
public:
```

```
    Trackable() : trackingID("NONE"), status("Idle") {}  
    Trackable(string tID) : trackingID(tID), status("Idle") {}  
    virtual ~Trackable() {}
```

```
void updateStatus(const string &newStatus) { status =  
newStatus; }
```

```
string getTrackingID() const { return trackingID; }
```

```

    string getStatus() const { return status; }
};

// ===== AMBULANCE (DIAMOND
// INHERITANCE) =====

class Ambulance : public Mobile, public Trackable {
private:
    int capacity;        // max patients per trip
    bool hasVentilator;
    int unitsInUse;      // 0 or capacity

public:
    Ambulance()           // default
    constructor

        : Resource(), Mobile(), Trackable(), capacity(0),
        hasVentilator(false), unitsInUse(0) {}

    Ambulance(string id, float speed, string loc, string trackID, //
    parameterized constructor
        int cap, bool ventilator)

        : Resource(id, "Ambulance"), Mobile(speed, loc),
        Trackable(trackID),
        capacity(cap), hasVentilator(ventilator), unitsInUse(0) {}

```

```
Ambulance(const Ambulance &a)                // copy  
constructor
```

```
    : Resource(a), Mobile(a), Trackable(a),  
      capacity(a.capacity), hasVentilator(a.hasVentilator),  
unitsInUse(a.unitsInUse) {}
```

```
Ambulance& operator=(const Ambulance &a) = default;  
// copy assignment
```

```
~Ambulance() {}                                // destructor
```

```
void allocate() override {  
    isAvailable = false;  
    unitsInUse = capacity;  
    updateStatus("On Trip");  
}
```

```
void release() override {  
    isAvailable = true;  
    unitsInUse = 0;  
    updateStatus("Idle");  
}
```

```
float calculateUtilization() const override {  
    if (capacity == 0) return 0.0f;
```

```

    return (static_cast<float>(unitsInUse) / capacity) * 100.0f;
}

```

```

void displayInfo() const override {
    cout << "[Ambulance] ID: " << resourceID
        << " | Location: " << currentLocation
        << " | Speed: " << speed << " km/h"
        << " | Capacity: " << capacity
        << " | Ventilator: " << (hasVentilator ? "Yes" : "No")
        << " | Status: " << status
        << " | Available: " << (isAvailable ? "Yes" : "No")
        << " | Utilization: " << fixed << setprecision(1)
        << calculateUtilization() << "%" << endl;
}

```

```

int getCapacity() const { return capacity; }

```

// Operator overload: combine the passenger capacity of two ambulances

```

Ambulance operator+(const Ambulance &other) const {
    Ambulance combined = *this;           // uses copy
    constructor
    combined.capacity = this->capacity + other.capacity;
    return combined;
}

```

```
    }  
};  
  
// ===== BED =====
```

```
class Bed : public Resource {  
private:  
    string ward;  
    bool hasOxygenSupport;  
    int unitsInUse;    // 0 or 1 (capacity = 1)  
  
public:  
    Bed() : Resource(), ward("General"),  
hasOxygenSupport(false), unitsInUse(0) {}  
  
    Bed(string id, string w, bool oxygen)  
        : Resource(id, "Bed"), ward(w), hasOxygenSupport(oxygen),  
unitsInUse(0) {}  
  
    Bed(const Bed &b)  
        : Resource(b), ward(b.ward),  
hasOxygenSupport(b.hasOxygenSupport),  
unitsInUse(b.unitsInUse) {}  
};
```

```
    Bed& operator=(const Bed &b) = default;           // copy
assignment
```

```
~Bed() {}
```

```
void allocate() override { isAvailable = false; unitsInUse = 1; }
```

```
void release() override { isAvailable = true; unitsInUse = 0; }
```

```
float calculateUtilization() const override {
```

```
    return unitsInUse * 100.0f;
```

```
}
```

```
void displayInfo() const override {
```

```
    cout << "[Bed] ID: " << resourceID
```

```
        << " | Ward: " << ward
```

```
        << " | Oxygen Support: " << (hasOxygenSupport ? "Yes"
: "No")
```

```
        << " | Available: " << (isAvailable ? "Yes" : "No")
```

```
        << " | Utilization: " << fixed << setprecision(1)
```

```
        << calculateUtilization() << "%" << endl;
```

```
}
```

```
};
```

```
// ===== EQUIPMENT
```

```
=====
```



```

class Equipment : public Resource {
private:
    string equipmentName;
    int totalUnits;
    int unitsAllocated;

public:
    Equipment() : Resource(), equipmentName("Unknown"),
totalUnits(0), unitsAllocated(0) {}

    Equipment(string id, string name, int units)
        : Resource(id, "Equipment"), equipmentName(name),
totalUnits(units), unitsAllocated(0) {}

    Equipment(const Equipment &e)
        : Resource(e), equipmentName(e.equipmentName),
totalUnits(e.totalUnits), unitsAllocated(e.unitsAllocated) {}

    Equipment& operator=(const Equipment &e) = default;    //
copy assignment

    ~Equipment() {}

    void allocate() override {

```

```
    if (unitsAllocated < totalUnits) unitsAllocated++;  
    if (unitsAllocated >= totalUnits) isAvailable = false;  
}
```

```
void release() override {  
    if (unitsAllocated > 0) unitsAllocated--;  
    isAvailable = true;  
}
```

```
float calculateUtilization() const override {  
    if (totalUnits == 0) return 0.0f;  
    return (static_cast<float>(unitsAllocated) / totalUnits) *  
100.0f;  
}
```

```
void displayInfo() const override {  
    cout << "[Equipment] ID: " << resourceID  
        << " | Name: " << equipmentName  
        << " | Units In Use: " << unitsAllocated << "/" <<  
totalUnits  
        << " | Available: " << (isAvailable ? "Yes" : "No")  
        << " | Utilization: " << fixed << setprecision(1)  
        << calculateUtilization() << "%" << endl;  
}
```

```
};
```

```
// ===== RESPONSE TEAM  
=====
```

```
class ResponseTeam : public Mobile {
```

```
private:
```

```
    int teamSize;
```

```
    string specialization;
```

```
    int unitsInUse;
```

```
public:
```

```
    ResponseTeam()
```

```
        : Resource(), Mobile(), teamSize(0),  
specialization("General"), unitsInUse(0) {}
```

```
    ResponseTeam(string id, float speed, string loc, int size, string  
spec)
```

```
        : Resource(id, "ResponseTeam"), Mobile(speed, loc),  
teamSize(size), specialization(spec), unitsInUse(0) {}
```

```
    ResponseTeam(const ResponseTeam &r)
```

```
        : Resource(r), Mobile(r), teamSize(r.teamSize),  
specialization(r.specialization), unitsInUse(r.unitsInUse) {}
```

```
ResponseTeam& operator=(const ResponseTeam &r) =  
default; // copy assignment
```

```
~ResponseTeam() {}
```

```
void allocate() override { isAvailable = false; unitsInUse =  
teamSize; }
```

```
void release() override { isAvailable = true; unitsInUse = 0; }
```

```
float calculateUtilization() const override {  
    if (teamSize == 0) return 0.0f;  
    return (static_cast<float>(unitsInUse) / teamSize) * 100.0f;  
}
```

```
void displayInfo() const override {  
    cout << "[ResponseTeam] ID: " << resourceID  
        << " | Specialization: " << specialization  
        << " | Team Size: " << teamSize  
        << " | Location: " << currentLocation  
        << " | Available: " << (isAvailable ? "Yes" : "No")  
        << " | Utilization: " << fixed << setprecision(1)  
        << calculateUtilization() << "%" << endl;  
}  
};
```

```
// ===== EMERGENCY CASE
=====
```

```
class EmergencyCase {
private:
    string caseID;
    Patient patient;
    int priorityLevel;          // 1 = critical ... 5 = minor
    Resource* assignedResource[MAX_CASE_RESOURCES];
// does NOT own these pointers
    int resourceCount;
    bool isActive;
    static int activeCases;     // static member: currently active
cases

public:
    EmergencyCase()             // default
constructor
        : caseID("NONE"), patient(), priorityLevel(5),
resourceCount(0), isActive(false) {
        for (int i = 0; i < MAX_CASE_RESOURCES; i++)
assignedResource[i] = nullptr;
    }
}
```

```
EmergencyCase(string id, const Patient &p, int priority) //  
parameterized constructor
```

```
    : caseID(id), patient(p), priorityLevel(priority),  
    resourceCount(0), isActive(true) {  
        for (int i = 0; i < MAX_CASE_RESOURCES; i++)  
            assignedResource[i] = nullptr;  
        activeCases++;  
    }
```

```
EmergencyCase(const EmergencyCase &c) //  
copy constructor
```

```
    : caseID(c.caseID), patient(c.patient),  
    priorityLevel(c.priorityLevel),  
    resourceCount(c.resourceCount), isActive(false) { //  
copies are inactive snapshots  
        for (int i = 0; i < MAX_CASE_RESOURCES; i++)  
            assignedResource[i] = c.assignedResource[i];  
    }
```

```
~EmergencyCase() { // destructor  
    if (isActive) {  
        for (int i = 0; i < resourceCount; i++) {  
            if (assignedResource[i] != nullptr) assignedResource[i]-  
>release();  
        }  
    }
```

```
}
```

```
EmergencyCase& operator=(const EmergencyCase &c) =  
default; // copy assignment
```

```
string getCaseID() const { return caseID; }
```

```
int getPriorityLevel() const { return priorityLevel; }
```

```
bool getIsActive() const { return isActive; }
```

```
bool assignResource(Resource* r) {
```

```
    if (resourceCount >= MAX_CASE_RESOURCES) return  
false;
```

```
    assignedResource[resourceCount++] = r;
```

```
    return true;
```

```
}
```

```
void updateCase(int newPriority) { priorityLevel =  
newPriority; }
```

```
void closeCase() {
```

```
    if (isActive) {
```

```
        for (int i = 0; i < resourceCount; i++) {
```

```
            if (assignedResource[i] != nullptr) assignedResource[i]-  
>release();
```

```
        }
```

```
        isActive = false;
        activeCases--;
    }
}
```

// Operator overload: priority comparison (lower number = higher priority)

```
bool operator<(const EmergencyCase &other) const {
    return priorityLevel < other.priorityLevel;
}
```

// Operator overload: formatted report output

```
friend ostream& operator<<(ostream &out, const
EmergencyCase &c) {
    out << "Case ID: " << c.caseID
        << " | Patient: " << c.patient.getName()
        << " | Priority: " << c.priorityLevel
        << " | Status: " << (c.isActive ? "Active" : "Closed")
        << " | Resources Assigned: " << c.resourceCount;
    return out;
}
```

```
static int getActiveCases() { return activeCases; }
};
```



```
int EmergencyCase::activeCases = 0;
```

```
// ===== HOSPITAL SYSTEM  
(MANAGER CLASS) =====
```

```
class HospitalSystem {
```

```
private:
```

```
    Patient patientList[MAX_PATIENTS];
```

```
    int patientCount;
```

```
    Resource* resourcePool[MAX_RESOURCES]; // base-class  
pointer array -> polymorphism
```

```
    int resourceCount;
```

```
    EmergencyCase caseList[MAX_CASES];
```

```
    int caseCount;
```

```
int findPatientIndex(const string &id) const {
```

```
    for (int i = 0; i < patientCount; i++)
```

```
        if (patientList[i].getPatientID() == id) return i;
```

```
    return -1;
```

```
}
```

```
int findCaseIndex(const string &id) const {
```

```

    for (int i = 0; i < caseCount; i++)
        if (caseList[i].getCaseID() == id) return i;
    return -1;
}

```

```

int findAvailableResourceIndex(const string &reqType) const
{
    for (int i = 0; i < resourceCount; i++) {
        if (resourcePool[i]->getType() == reqType &&
            resourcePool[i]->getAvailability())
            return i;
    }
    return -1;
}

```

public:

```

    HospitalSystem() : patientCount(0), resourceCount(0),
        caseCount(0) {
        for (int i = 0; i < MAX_RESOURCES; i++) resourcePool[i]
            = nullptr;
    }

```

```

    ~HospitalSystem() {
        // Close any still-active cases first so they release their
        assigned
    }

```

// resources while those resources are still valid. This must happen

// before the resources themselves are deleted below, and before

// caseList's own elements are automatically destroyed (which would

// otherwise try to release() resources that no longer exist).

```
for (int i = 0; i < caseCount; i++) {
```

```
    if (caseList[i].getIsActive()) caseList[i].closeCase();
```

```
}
```

```
for (int i = 0; i < resourceCount; i++) delete resourcePool[i];
```

```
}
```

```
void registerPatient() {
```

```
    if (patientCount >= MAX_PATIENTS) { cout << "Patient list is full.\n"; return; }
```

```
    string id = readLine("Enter Patient ID: ");
```

```
    string name = readLine("Enter Patient Name: ");
```

```
    int age = readIntInRange("Enter Age: ", 0, 130);
```

```
    string condition = readLine("Enter Emergency Condition: ");
```

```
    Patient newPatient(id, name, age, condition); //  
parameterized constructor
```

```
    Patient auditCopy(newPatient);           // copy  
    constructor (audit record)
```

```
    patientList[patientCount++] = newPatient;
```

```
    cout << "Patient registered successfully.\n";
```

```
    cout << "Audit copy on file -> ";
```

```
    auditCopy.displayPatient();
```

```
    cout << "Total patients registered so far: " <<  
    Patient::getTotalPatients() << endl;  
}
```

```
void registerResource() {
```

```
    if (resourceCount >= MAX_RESOURCES) { cout <<  
    "Resource pool is full.\n"; return; }
```

```
    cout << "\nSelect Resource Type:\n1. Ambulance\n2.  
Bed\n3. Equipment\n4. Response Team\n";
```

```
    int typeChoice = readIntInRange("Enter choice: ", 1, 4);
```

```
    string id = readLine("Enter Resource ID: ");
```

```
    Resource* newResource = nullptr;
```

```
    if (typeChoice == 1) {
```

```
        float speed = readFloat("Enter speed (km/h): ");
```

```
        string loc = readLine("Enter current location: ");
```

```

        string trackID = readLine("Enter tracking ID: ");
        int capacity = readIntInRange("Enter patient capacity: ",
1, 10);

        int vent = readIntInRange("Has ventilator? (1=Yes,
0=No): ", 0, 1);

        newResource = new Ambulance(id, speed, loc, trackID,
capacity, vent == 1);

    } else if (typeChoice == 2) {
        string ward = readLine("Enter ward name: ");
        int oxy = readIntInRange("Has oxygen support? (1=Yes,
0=No): ", 0, 1);
        newResource = new Bed(id, ward, oxy == 1);
    } else if (typeChoice == 3) {
        string name = readLine("Enter equipment name: ");
        int units = readIntInRange("Enter total units available: ",
1, 100);
        newResource = new Equipment(id, name, units);
    } else {
        float speed = readFloat("Enter speed (km/h): ");
        string loc = readLine("Enter current location: ");
        int size = readIntInRange("Enter team size: ", 1, 20);
        string spec = readLine("Enter specialization: ");
        newResource = new ResponseTeam(id, speed, loc, size,
spec);
    }

```

```

    resourcePool[resourceCount++] = newResource;
    cout << "Resource registered successfully.\n";
    cout << "Total resources registered so far: " <<
Resource::getTotalResources() << endl;
}

```

```

void createEmergencyCase() {
    if (caseCount >= MAX_CASES) { cout << "Case list is
full.\n"; return; }

    if (patientCount == 0) { cout << "No patients registered
yet.\n"; return; }

```

```

    string patientID = readLine("Enter Patient ID for this case:
");

```

```

    int pIndex = findPatientIndex(patientID);
    if (pIndex == -1) { cout << "Patient not found.\n"; return; }

```

```

    string caseID = readLine("Enter Case ID: ");
    int priority = readIntInRange("Enter priority level
(1=Critical ... 5=Minor): ", 1, 5);

```

```

    EmergencyCase newCase(caseID, patientList[pIndex],
priority); // parameterized constructor

```

```

    EmergencyCase auditSnapshot(newCase); //
copy constructor (audit)

```

```

    caseList[caseCount++] = newCase;

    cout << "Emergency case created successfully.\n";
    cout << "Audit snapshot -> " << auditSnapshot << endl;
    cout << "Total active cases: " <<
EmergencyCase::getActiveCases() << endl;
}

void displayPatients() const {
    if (patientCount == 0) { cout << "No patients registered.\n";
return; }

    cout << "\n--- Registered Patients (" << patientCount << ") -
--\n";

    for (int i = 0; i < patientCount; i++)
patientList[i].displayPatient();
}

void displayResources() const {
    if (resourceCount == 0) { cout << "No resources
registered.\n"; return; }

    cout << "\n--- Registered Resources (" << resourceCount <<
") ---\n";

    for (int i = 0; i < resourceCount; i++) resourcePool[i]-
>displayInfo(); // polymorphism
}

```

```

void allocateResource() {
    if (caseCount == 0) { cout << "No emergency cases
exist.\n"; return; }

    string caseID = readLine("Enter Case ID to allocate a
resource to: ");

    int cIndex = findCaseIndex(caseID);
    if (cIndex == -1) { cout << "Case not found.\n"; return; }
    if (!caseList[cIndex].getIsActive()) { cout << "This case is
closed.\n"; return; }

    cout << "\nSelect Required Resource Type:\n1.
Ambulance\n2. Bed\n3. Equipment\n4. Response Team\n";
    int typeChoice = readIntInRange("Enter choice: ", 1, 4);
    string requiredType = (typeChoice == 1) ? "Ambulance" :
        (typeChoice == 2) ? "Bed" :
        (typeChoice == 3) ? "Equipment" :
"ResponseTeam";

    int rIndex = findAvailableResourceIndex(requiredType);
    if (rIndex == -1) { cout << "No available " << requiredType
<< " at this time.\n"; return; }

    resourcePool[rIndex]->allocate();                // virtual
dispatch

```



```

        caseList[cIndex].assignResource(resourcePool[rIndex]);

        cout << requiredType << " (" << resourcePool[rIndex]-
>getResourceID()
        << ") allocated successfully to case " << caseID << ".\n";
    }

void updateCaseStatus() {
    if (caseCount == 0) { cout << "No emergency cases
exist.\n"; return; }

    string caseID = readLine("Enter Case ID to update: ");
    int cIndex = findCaseIndex(caseID);
    if (cIndex == -1) { cout << "Case not found.\n"; return; }

    cout << "1. Update priority level\n2. Close case\n";
    int choice = readIntInRange("Enter choice: ", 1, 2);
    if (choice == 1) {
        int newPriority = readIntInRange("Enter new priority
level (1=Critical ... 5=Minor): ", 1, 5);
        caseList[cIndex].updateCase(newPriority);
        cout << "Case priority updated.\n";
    } else {
        caseList[cIndex].closeCase();
        cout << "Case closed. Assigned resources released.\n";
    }
}

```

```
}
```

```
void comparePriorities() {  
    if (caseCount < 2) { cout << "Need at least two cases to  
compare.\n"; return; }  
    string id1 = readLine("Enter first Case ID: ");  
    string id2 = readLine("Enter second Case ID: ");  
    int i1 = findCaseIndex(id1);  
    int i2 = findCaseIndex(id2);  
    if (i1 == -1 || i2 == -1) { cout << "One or both cases not  
found.\n"; return; }  
  
    cout << "Case 1 -> " << caseList[i1] << endl;  
    cout << "Case 2 -> " << caseList[i2] << endl;  
  
    if (caseList[i1] < caseList[i2])  
        cout << caseList[i1].getCaseID() << " has higher priority  
(needs attention first).\n";  
    else if (caseList[i2] < caseList[i1])  
        cout << caseList[i2].getCaseID() << " has higher priority  
(needs attention first).\n";  
    else  
        cout << "Both cases have equal priority.\n";  
}
```

```

void calculateUtilization() const {
    if (resourceCount == 0) { cout << "No resources
registered.\n"; return; }

    cout << "\n--- Resource Utilization ---\n";

    for (int i = 0; i < resourceCount; i++) {
        cout << resourcePool[i]->getResourceID() << " (" <<
resourcePool[i]->getType() << "): "
            << fixed << setprecision(1) << resourcePool[i]-
>calculateUtilization() << "%\n";
    }
}

```

```

void generateReport() const {
    cout << "\n===== HOSPITAL COORDINATION REPORT
===== \n";

    cout << "Total Patients Registered : " <<
Patient::getTotalPatients() << endl;

    cout << "Total Resources Registered: " <<
Resource::getTotalResources() << endl;

    cout << "Currently Active Cases   : " <<
EmergencyCase::getActiveCases() << endl;

    cout << "\n-- Resource Status --\n";

    for (int i = 0; i < resourceCount; i++) resourcePool[i]-
>displayInfo();
}

```

```

        cout << "\n-- Case Summary --\n";
        for (int i = 0; i < caseCount; i++) cout << caseList[i] <<
endl;

        cout <<
"=====\\n";
    }

```

```

void combineAmbulances() {
    cout << "This combines the passenger capacity of two
ambulances (operator+ demo).\\n";

    string id1 = readLine("Enter first Ambulance ID: ");
    string id2 = readLine("Enter second Ambulance ID: ");

    int i1 = -1, i2 = -1;
    for (int i = 0; i < resourceCount; i++) {
        if (resourcePool[i]->getType() == "Ambulance" &&
resourcePool[i]->getResourceID() == id1) i1 = i;

        if (resourcePool[i]->getType() == "Ambulance" &&
resourcePool[i]->getResourceID() == id2) i2 = i;
    }

    if (i1 == -1 || i2 == -1) { cout << "One or both ambulances
not found.\\n"; return; }
}

```

```

    Ambulance* a1 =
dynamic_cast<Ambulance*>(resourcePool[i1]);

```

```

        Ambulance* a2 =
dynamic_cast<Ambulance*>(resourcePool[i2]);

        if (a1 == nullptr || a2 == nullptr) { cout << "Type
mismatch.\n"; return; }

        Ambulance combined = (*a1) + (*a2); // operator+ usage
        cout << "Combined capacity: " << combined.getCapacity()
<< " patients.\n";
    }

```

```

void showMenu() const {
    cout << "\n===== SMART EMERGENCY
MEDICAL RESOURCE COORDINATION SYSTEM
=====\\n";

    cout << "1. Register Patient\\n";
    cout << "2. Register Emergency Resource\\n";
    cout << "3. Create Emergency Case\\n";
    cout << "4. Display Patients\\n";
    cout << "5. Display Resources\\n";
    cout << "6. Allocate Resource to Case\\n";
    cout << "7. Update / Close Emergency Case\\n";
    cout << "8. Compare Case Priorities\\n";
    cout << "9. Calculate Resource Utilization\\n";
    cout << "10. Generate Coordination Report\\n";
}

```

```

        cout << "11. Combine Two Ambulances (operator+
demo)\n";

        cout << "0. Exit\n";

        cout <<
"=====
=====\\n";

    }

```

```

void run() {
    int choice;
    do {
        showMenu();
        choice = readIntInRange("Enter your choice: ", 0, 11);
        switch (choice) {
            case 1: registerPatient(); break;
            case 2: registerResource(); break;
            case 3: createEmergencyCase(); break;
            case 4: displayPatients(); break;
            case 5: displayResources(); break;
            case 6: allocateResource(); break;
            case 7: updateCaseStatus(); break;
            case 8: comparePriorities(); break;
            case 9: calculateUtilization(); break;
            case 10: generateReport(); break;
            case 11: combineAmbulances(); break;

```

```

        case 0: cout << "Exiting system. Goodbye!\n"; break;
    }
    } while (choice != 0);
}

};

// ===== MAIN =====

int main() {
    HospitalSystem system;
    system.run();
    return 0;
}

```

Test Cases:

TC No.	Test Case	Input	Expected Result	Actual Result
TC01	Patient registration	Valid patient details	Patient registered	Registered, Total count=1
TC02	Resource registration	Ambulance details	Resource registered	Registered, Total count=1
TC03	Case creation	Valid emergency details	Case created	Created, active cases=1
TC04	Priority assignment	Critical case	High priority assigned	Stored as 1
TC05	Resource allocation	Available ambulance	Ambulance allocated	Ambulance A1 allocated successfully to case C1

TC No.	Test Case	Input	Expected Result	Actual Result
TC06	Unavailable resource	Occupied ambulance	Allocation rejected	No available ambulance at this time
TC07	Case comparison	Two cases	Higher priority identified	C1 has higher priority
TC08	Case completion	Completed case	Resource released	Case closed. Assigned resources released." — confirmed the ambulance becomes available again
TC09	Utilization	Used resources	Correct utilization shown	20.% shown
TC10	Report generation	Report option	Coordination report displayed	Shows totals, resource status, case summary
TC11	Invalid menu	Invalid number	Error message	Invalid input" / "Please enter a value between X and Y
TC12	Exit	0	Program terminates	Exit code 0, no memory errors (verified with AddressSanitizer)

Implementation Screenshots:

- Figure 8.1: Main menu of the Smart Emergency Medical Resource Coordination System.


```

===== SMART EMERGENCY MEDICAL RESOURCE COORDINATION SYSTEM =====
1. Register Patient
2. Register Emergency Resource
3. Create Emergency Case
4. Display Patients
5. Display Resources
6. Allocate Resource to Case
7. Update / Close Emergency Case
8. Compare Case Priorities
9. Calculate Resource Utilization
10. Generate Coordination Report
11. Combine Two Ambulances (operator+ demo)
0. Exit
=====
Enter your choice: 

```

- Figure 8.2: Patient registration.

```

Enter your choice: 1
Enter Patient ID: P001
Enter Patient Name: Shahana
Enter Age: 19
Enter Emergency Condition: Asthma
Patient registered successfully.
Audit copy on file -> Patient ID: P001 | Name: Shahana | Age: 19 | Condition: Asthma
Total patients registered so far: 1

```

- Figure 8.3: Emergency-resource registration.

```

Enter Resource ID: A1
Enter speed (km/h): 80
Enter current location: City Hospital
Enter tracking ID: T1
Enter patient capacity: 2
Has ventilator? (1=Yes, 0=No): 1
Resource registered successfully.
Total resources registered so far: 1

```

- Figure 8.4: Emergency-case creation.

```

=====
Enter your choice: 3
Enter Patient ID for this case: P001
Enter Case ID: C001
Enter priority level (1=Critical ... 5=Minor): 1
Emergency case created successfully.
Audit snapshot -> Case ID: C001 | Patient: Shahana | Priority: 1 | Status: Closed | Resources Assigned: 0
Total active cases: 1

```

- Figure 8.5: Priority evaluation.

```

Enter your choice: 7
Enter Case ID to update: C003
1. Update priority level
2. Close case
Enter choice: 1
Enter new priority level (1=Critical ... 5=Minor): 1
Case priority updated.

```

- Figure 8.6: Resource allocation.

```

Select Required Resource Type:
1. Ambulance
2. Bed
3. Equipment
4. Response Team
Enter choice: 1
Ambulance (A1) allocated successfully to case C001.

```

- Figure 8.7: Resource status update.

```

Enter your choice: 7
Enter Case ID to update: C002
1. Update priority level
2. Close case
Enter choice: 2
Case closed. Assigned resources released.

```

- Figure 8.8: Case-priority comparison.

```

Enter your choice: 8
Enter first Case ID: C001
Enter second Case ID: C003
Case 1 -> Case ID: C001 | Patient: Shahana | Priority: 1 | Status: Active | Resources Assigned: 0
Case 2 -> Case ID: C003 | Patient: Bala | Priority: 2 | Status: Active | Resources Assigned: 0
C001 has higher priority (needs attention first).

```

- Figure 8.9: Resource utilization.

```

Enter your choice: 9

--- Resource Utilization ---
R1 (Ambulance): 0.0%

```

- Figure 8.10: Final coordination report.

```

[Ambulance] ID: R1 | Location: City Hospital | Speed: 90.0 km/h | Capacity: 6 | Ventilator: Yes | Status: Idle | Available: Yes | Utilization: 0.0%

-- Case Summary --
Case ID: C001 | Patient: Shahana | Priority: 1 | Status: Active | Resources Assigned: 0
Case ID: C001 | Patient: Shri | Priority: 5 | Status: Active | Resources Assigned: 0
Case ID: C003 | Patient: Bala | Priority: 2 | Status: Active | Resources Assigned: 0
=====

```

- Figure 8.11: Polymorphism / resource display.

```

Enter your choice: 5

--- Registered Resources (1) ---
[Ambulance] ID: R1 | Location: City Hospital | Speed: 90.0 km/h | Capacity: 6 | Ventilator: Yes | Status: Idle | Available: Yes | Utilization: 0.0%

```

- Figure 8.12: Exit / termination.

```

Enter your choice: 0
Exiting system. Goodbye!

...Program finished with exit code 0
Press ENTER to exit console.

```

Individual Contribution of Group Members

Team Member	Contribution
Shahana	Designed and implemented the Patient and EmergencyCase classes, including patient registration, case creation, encapsulation, member functions, static members, arrays of objects, and priority management.
Rasineni Harika	Designed and implemented the Resource hierarchy, including Mobile, Trackable, Ambulance, Bed, Equipment, and ResponseTeam. Worked on inheritance, the abstract class, the virtual base class, constructors, and destructors.
Niranjana. A	Integrated the complete application and implemented resource allocation, pointers, runtime polymorphism, operator overloading, menu-driven functionality, utilization calculation, report generation, testing, and GitHub upload.